MODULE	**ITC 4214 – INTERNET PROGRAMMING (LEVEL 6)**		
TERM	**FALL TERM 2026**		
ASSESSMENT	**COURSEWORK**	**WEIGHT**	**30%**
RELEVANT LEARNING OUTCOMES	1. Develop programming skills in client-side and server-side languages for the web. 2. Demonstrate understanding of the functionality and evaluate the effectiveness of web development libraries and frameworks, as well as integrate their features in a web site.		
DELIVERABLES AND METHOD OF SUBMISSION	A report that will include the URL of your GitHub repository Zip file of sources uploaded on BLACKBOARD		
PROFESSOR NAME	Leonardos Mageiros		
DATE	***Regular submission: TBA*** ***Late submission: TBA*** **Project Presentations: TBA** *Work submitted late and within 6 working days after the deadline will be penalized by 10% for each day of delay to a minimum of **40** points (for passing grades). Students will fail the assessment if work is submitted after the late submission deadline.*		

College regulations regarding academic integrity and offences stand, as they are described in the Undergraduate Catalog & Student Handbook, pp 52-56.

The assignment is performed individually. Your work should be performed by using a GitHub repository. Your commit points should have meaningful messages and must be submitted in regular time intervals.

Specifications:

You will create a task management application for a company of your choice where users can manage their tasks. The application will have five pages: Home, Tasks, About, Contact, and one page of your choice. All DOM manipulation should be done using jQuery or vanilla JavaScript. The HTML should be well-formed with appropriate metadata, and the website should be fully responsive.

Page Requirements:

A) Home Page (index.html):

1. **Welcome Section:**
   - A brief introduction about the company.
   - An overview of the products and features.
   - A call-to-action button that directs users to the Tasks page.
2. **Latest Activity:**
   - A section that shows the latest activity on tasks (e.g., tasks added, completed).
3. **Navigation Bar:**
   - A sticky header with the application logo, page title, and navigation links to other sections/pages.
4. **API integration:**
   - Add a feature that fetches data from a public API and displays it dynamically on one of the pages (for instance, motivational quotes or weather updates).
5. **Dark Mode:**
   - Implement a dark mode toggle that allows users to switch between light and dark color schemes.

B) Tasks Page (tasks.html):

1. **Task Management:**
   - Implement a persistent storage of tasks that should include the following.
   - A form to add new tasks, including fields for the task name, description, and due date.
   - A table or list of tasks showing the task name, description, due date, and status.
   - Edit and delete buttons for each task.
   - A button to mark tasks as completed.
   - Add a section showing a summary of total tasks, pending tasks, and completed tasks.
   - Bonus: Allow users to assign priorities (e.g., High, Medium, Low) to tasks. Color code each priority level and enable filtering by priority.
2. **Filtering and Sorting:**
   - Filtering feature to view tasks by status (e.g., all, completed, pending).
   - Sorting feature to sort tasks by due date or task name.
   - Implement also a live search field that filters tasks by title or description as the user types.

3. **Task analytic Dashboard:**
    - Create a small dashboard (you could put it on the Home page or a new page called analytics.html) that visualizes task data — such as a bar chart showing completed vs. pending tasks, or pie chart by priority.
    - Here create also a Kanban-style task board that display your tasks in three columns (To do, in progress and completed).
    - Each task should appear as a card and users should be able to move tasks between columns. Doing so should automatically update their status.

C) About Page (about.html):

1. **About Section:**
    - Information about the application, its purpose, and its features.
2. **Team Members:**
    - A section to display information about team members with avatars, names, and roles. If you do not have a team, pretend that you do!
3. **Grid Cards:**
    - A grid layout with cards showing team members' names, avatars, and short info.
4. **Carousel:**
    - A carousel with images or testimonials related to the application or development process. If you like you can put the carousel in the home page as well.

D) Contact Page (contact.html):

1. **Contact Form:**
    - A form that collects the user's name, email, subject, and message. Use appropriate validation of the fields.
    - When submitted, the details should be captured and appear as a confirmation pop-up on the screen.
2. **Google Maps Integration:**
    - Embed a Google Map showing the location of the team or the office.
3. **Social Media Links:**
    - Icons linking to the application's or team members' social media profiles.

E) Page of your choice

- The page should be an html file named according to its context.
- It should display the same header/footer with the rest of the website.
- It should include 3 new components of your choice.

General Requirements for All Pages:

1. **CSS Framework:**
    - Use a CSS framework of your choice (e.g., Bootstrap) to style the website.
2. **Responsive Design:**

- Ensure the website is fully responsive and works well on all devices (desktop, tablet, mobile).

3. **Footer Section:**
   - A footer with the application logo, copyright information, and contact details (social media links, email).
4. **Accessibility:**
   - Ensure the website meets basic accessibility standards (e.g., proper use of ARIA labels, alt text for images, proper meta tags etc).

Additional Technical Requirements:

1. **Responsive Navigation:**
   - Implement a responsive navigation menu that collapses into a hamburger menu on smaller screens.
2. **Form Validation:**
   - Implement client-side form validation for all forms.
3. **Code Duplication**
   - Avoid code duplication an excessive use of global variables as much as possible
4. **Version Control:**
   - Use Git for version control with a repository hosted on GitHub, GitLab, or Bitbucket.
5. **Presentation:**
   Create a short presentation to demonstrate to your classmates, the basic structure, and components of the client-side framework, which you chose to implement for your assignment. You must be able to present your assignment in-class without a notice and explain your code in detail within a short notice after your submission to get a passing grade.

Be prepared to explain your code structure and the reasons behind your implementation decisions.

Report

Create a word document to include the following about your web site:

- Description of your task allocation system
- Detailed documentation of the coding decisions of your tasks page, latest activity section and of your personal page
- Github link
- Use Chrome Lighthouse or another auditing tool to test their site's **accessibility**, **SEO**, and **performance**, and include a short reflection with screenshots in your report.
- Reflections: reflect on the challenges you faced for this work and how you dealt with them.
- You can implement this as a word document or as a webpage in your site or a README.md for your github repository.

MARKING SCHEME

Criterion	Max Points
HTML home page structure and good programming practices (eg comments, metatags, clear coding, no repeated code etc)	7
Form with fields and add button which adds tasks in a list	7
HTML table which displays tasks from a JavaScript list and analytics of tasks section.	8
Delete and edit Feature	8
Searching, Sorting and Filtering Feature	7
Overall JavaScript and jQuery code structure, modularity, documentation	8
About and contact me page - overall formatting and content	5
Appropriate use of CSS Framework and responsiveness	8
Sticky Header, Logo, Title, Sandwich Menu, dark mode	5
Footer	5
Latest activity section and API integration	8
Appropriate Use of GitHub, where version control is obvious	7

Page of your choice and Kanban board	7
Report and performance audit	7
Personal reflections	3